ThereMen: Accessible Theremin

Zachary Logeman
Maximilian Nobrega
Luke Hamamji
Danny Sager

Team Name; ECEN 1400 - 024

November 19, 2025

Abstract

The accessible theremin is a modified version of a theremin, an electronic instrument where the pitch and volume are controlled by the location and movement of your hands around electromagnetic rods. Since the theremin is played in open air, notes can be ambiguous, making the theremin a difficult instrument to learn. Our project solves this problem by including a screen that tells the user which note they are currently playing. We utilize two distance sensors (in place of a traditional theremin's rods) that read the location of the user's hands, which is then sent to the speakers to produce a tone and to the screen to display the current note. Our goal is to make the theremin easier and more accessible for new players.

Contents

1. Introduction 4
   1.1. Project Overview 4
   1.2. Client 4
   1.3. Inspiration 5
2. Project Structure 5
   2.1. Team Assignments 5
   2.2. Goal 5
3. Specifications 6
4. Design Iteration 6
   4.1. Initial Design Ideas 7
   4.2. Prototyping and Design Loop 7
   4.3. Final Design Elements 7
   4.4. Risk Mitigation During Design 8
5. Electrical Design 8
   5.1. Block Diagram Electrical 8
   5.2. Circuits and Components 9
   5.3. Protoboard Design 9
   5.4. Electrical Function 10
6. Software Design 10
   6.1. Pseudo Code; Flow Charts 11
   6.2. Programming issues 12
7. Mechanical Design 12

7.1. Mechanical Components 13
7.2. Fabrication and Assembly 14
7.2.1. Tools 16
7.3. Mechanical Functions 16
8. Testing and Analysis 17
8.1. Electrical Testing 17
8.1.1. Qualitative Analysis 18
8.1.2. Quantitative Analysis 18
8.2. Mechanical Testing 18
8.2.1. Qualitative Analysis 19
8.2.2. Quantitative Analysis Pitch 19
8.3. Overall Results of Testing 19
9. Timeline and Bill of Materials. 20
9.1. Gantt Chart 20
9.2. Final Bill of materials (BOM) 21
10. Conclusion 21
10.1. Lessons Learned 21
10.2. Next Steps or Future Work 22

1. Introduction

Our project is a theremin-style electronic musical instrument that lets the player control pitch and volume just by moving their hands in the air. Using ultrasonic distance sensors and a microcontroller, the system measures hand position and turns it into musical notes played through a small speaker. The overall goal is to build a stable, easy-to-use prototype that clearly shows how motion can be converted into sound in real time. This matters for ECEN 1400 because it pulls together key course topics of sensors, embedded programming, and signal generation into one complete, working system instead of isolated lab exercises.

1.1. Project Overview

The main hardware components are a SparkFun RedBoard, two ultrasonic distance sensors (one for pitch and one for volume), a 16×2 LCD screen, a small speaker/amp, and supporting resistors, capacitors, and wiring mounted in a simple enclosure. The LCD displays the current note frequency so that players get clear visual feedback about what they are playing, even if they don't have a trained ear.

In operation, the player stands in front of the device and moves one hand over each sensor. The RedBoard measures the distance from each hand, maps the pitch sensor distance to a musical frequency and the volume sensor distance to loudness, and then uses PWM audio to drive the speaker. The software, written in Arduino C++, continuously reads both sensors, filters the measurements to reduce jitter, updates the LCD with the current frequency, and generates the corresponding tone. Key performance requirements include a clear pitch range, smooth volume control, low latency, readable LCD output, and reliable operation over repeated demos.

1.2. Client

The formal client for this project is the ECEN 1400 teaching staff, who defined the requirements and will grade the final prototype. A major requirement was that the device be accessible and easy for beginners to understand and use, which is why we added the LCD display to show the current frequency and make the notes more obvious to new players. A secondary client is the audience at the end-of-semester expo, who should be able to walk up, read the display, and quickly figure out how to interact with the instrument.

The client expects a safe, working embedded system that provides clear feedback to the user, and is well documented with schematics, code, and explanations. We worked within constraints such as a limited part budget, the requirement to use the Arduino-based platform, and the fixed semester schedule. Success will be evaluated by whether the instrument functions as intended, how well it demonstrates course concepts and accessibility, and the overall quality of our documentation and presentation.

1.3. Inspiration

This project was inspired by the original theremin, one of the first electronic instruments, which is played without physical contact by moving hands near two antennas. We wanted to recreate the same “invisible” control, but using modern, easier-to-use ultrasonic sensors instead of RF circuits. A theremin-style device is especially interesting because it makes abstract ideas about distance sensing, mapping, and audio synthesis very visible and intuitive for users. On a personal level, the project let us combine an interest in music with our growing skills in electronics, coding, CAD, and hands-on prototyping, making it a motivating way to apply what we learned in ECEN 1400.

2. Project Structure

Our team organized the project so that each person had a clear primary role and a secondary area where they could support others. This structure helped us divide the work on software, electronics, CAD, and woodworking while still keeping everyone involved in multiple parts of the building. We also set concrete goals for what we wanted to buy, what we needed to physically build, and which outside resources we would rely on, such as the ITLL labs and TA hours.

AI tools played a supporting role in our process. We used them mainly for brainstorming code structures, checking syntax, and getting ideas for debugging strategies or sensor mappings. However, all final design decisions, wiring choices, and code implementations were tested, modified, and verified by our team to make sure they met the project requirements and worked reliably on our hardware.

2.1. Team Assignments

Each team member had a main responsibility area and a backup role. Maximilian served as the main software lead, writing and refining most of the Arduino code and helping with electronics when needed. Luke focused on the woodworking and overall physical build of the enclosure while also contributing to CAD work and electronics assembly. Zach took the lead on CAD design for the 3D-printed parts, and supported the woodworking when adjustments were needed. Danny was responsible for the main electronics, including wiring, component selection, and troubleshooting, and acted as an additional software resource when the code and hardware needed to be debugged together.

2.2. Goal

From the start, we set a clear set of build goals. On the purchased side, we obtained ultrasonic sensors, the LCD display, an audio amplifier, and a speaker system to create the sensing and sound-output chain. On the built side, we designed and fabricated a 3D-printed

casing, created a wooden lid, and produced the electronic schematics and Arduino code that tie everything together.

We relied on several key resources to reach these goals: the woodwork shop, the 3-D printers, the soldering lab and component stock in the ITLL, assistance from TAs during lab hours, and access to different resistors and capacitors when our original choices did not behave as expected. Even with these resources, there were features we chose not to implement within the project scope. Our current prototype does not display notes in standard musical notation because the RedBoard can only handle one task at a time. Adding the extra calculations to convert frequency into note names (on top of sensor reading, audio generation, and LCD updates) caused the sound to cut out instead of staying continuous, so we prioritized a stable, responsive instrument that shows frequency only.

3. Specifications

Our theremin uses two 5 V ultrasonic distance sensors, one assigned to pitch and one to volume. The sensors are read by a SparkFun RedBoard, which converts the measured distance into a note frequency and a loudness level. Audio is generated with the Mozzi library on a PWM pin and fed through a small mono amplifier module that drives an 8 Ω speaker; we also left the option to connect to external powered speakers if needed.

Within the intended playing range of roughly 0–60 cm above each sensor, repeated readings were generally stable within a couple of centimeters, which was enough to keep the mapped notes from jumping around too much. To keep the interface accessible, we added a 16×2 LCD that shows the current frequency being played. The display is updated about twice per second, so it stays readable and doesn't slow down the audio loop. Because the frequency shown on the LCD is calculated from the same note table used by Mozzi, it closely matches the pitch the user actually hears.

4. Design Iteration

Our design went through several build–test–fix cycles before the final version. The first prototype was a simple breadboard with one ultrasonic sensor, a microcontroller, and a speaker. It proved we could read distance and make sound, but the pitch was noisy, the wiring was fragile, and there was no visual feedback.

In later iterations, we added a second sensor for volume, switched to Mozzi for smoother audio, and brought in the LCD. This revealed issues with jittery sensor data, slow updates, and unreliable breadboard connections. We fixed these by averaging readings, limiting the playable distance range, reducing the LCD update rate, and moving to soldered perfboard in a 3D-printed and wooden enclosure. These changes made the theremin more stable, easier to understand, and ready for live demos.

4.1. Initial Design Ideas

Our starting idea was a basic theremin where hand distance directly controls pitch and volume, one hand over the “pitch” sensor and one over the “volume” sensor. The initial block diagram was simple: two ultrasonic sensors feed distance data into the microcontroller, which then outputs audio to an amplifier and speaker while also sending information to an LCD. From the beginning, we wanted the instrument to feel approachable, not just like a technical demo.

We chose ultrasonic sensors because they are inexpensive, safe, and easy to interface with an Arduino, and they give a clear link between hand height and measured distance. For audio, we picked the Mozzi library instead of the basic `tone()` function because it supports better sound quality and non-blocking timing, which is important when we need to constantly read sensors. Early on, we brainstormed several ways to use the LCD: showing raw distance, note names, or frequency. We eventually focused on displaying the current frequency, since that gives players a concrete number to match and makes the theremin more accessible to people who don’t yet recognize notes by ear.

4.2. Prototyping and Design Loop

We followed an iterative build–test–fix loop with five main versions of the design. Iteration 1 focused on basic sensing: reading one ultrasonic sensor and checking that distance values were stable. Iteration 2 added audio output, first with simple tones and then with Mozzi so we could test the mapping from distance to pitch. In Iteration 3, we introduced the second sensor for volume control and tuned the ranges, so both hands felt natural to use. Iteration 4 integrated the LCD, which required adjusting update rates, so the screen stayed readable without slowing the audio. Iteration 5 finalized the physical layout and enclosure, moving from breadboard to soldered perfboard and mounting everything in the 3D-printed and wooden structure.

4.3. Final Design Elements

The final design is a self-contained theremin-style instrument with two ultrasonic sensors feeding distance data to a SparkFun RedBoard. The microcontroller processes these readings, maps them to pitch and volume, and uses Mozzi to generate audio on a PWM pin. That signal is amplified and sent to an 8 Ω speaker, with the option to connect external speakers if needed. A 16×2 LCD displays the current frequency, so players can see what note they are near, improving accessibility for beginners. All electronics are soldered to perfboard and mounted inside a 3D-printed casing with a wooden base, which routes power and wiring cleanly while holding the sensors at a fixed angle toward the player.

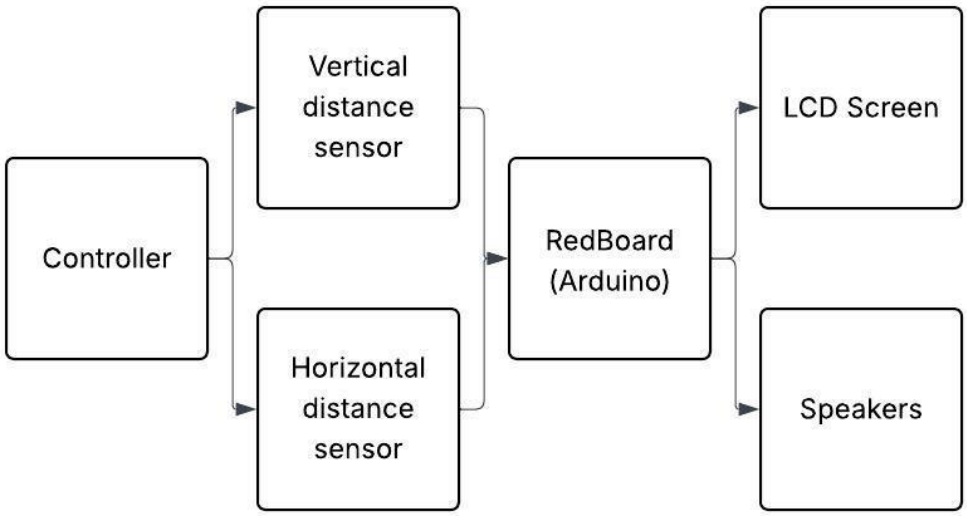
4.4. Risk Mitigation During Design

Because several design choices kept changing, especially which LCD and amplifier to use, we made the enclosure and internal layout as adaptable as possible. The 3D-printed parts include extra space and cable pass-throughs so we could reroute wires or swap modules without reprinting the entire box so many times. We also designed the system to run from either a battery pack or a wall adapter, reducing the risk of power issues during the expo. Finally, the code was written in a modular way, so when hardware changed (like different pin assignments or LCD behavior), we could update a few sections of software instead of rewriting the whole program.

5. Electrical Design

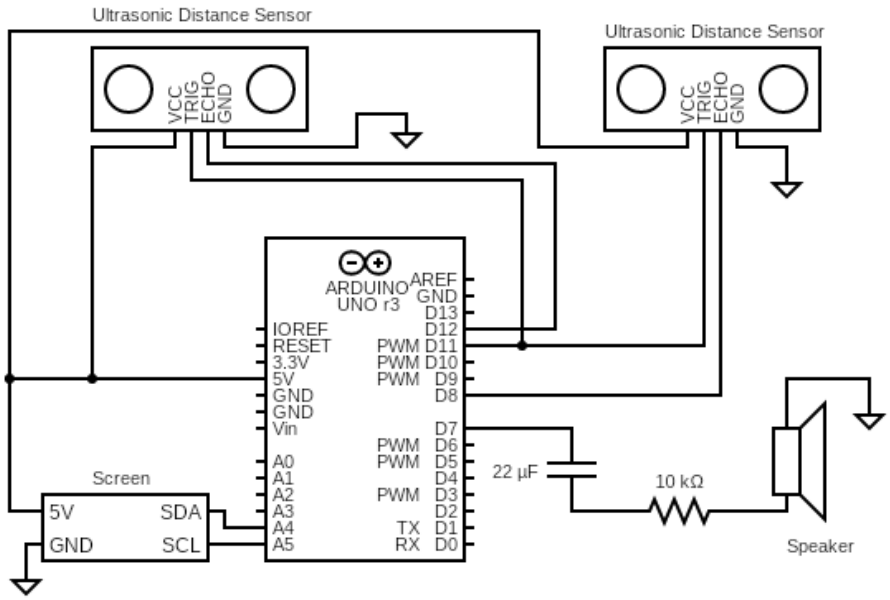
This section will review all components of our projects electrical design. Beginning with our basic block diagram, we will then explain our final circuit layout and design. We will then explain and show our protoboard. Lastly, we will explain the electrical function, with an emphasis on how different parts of our design are able to communicate with each other.

5.1. Block Diagram Electrical



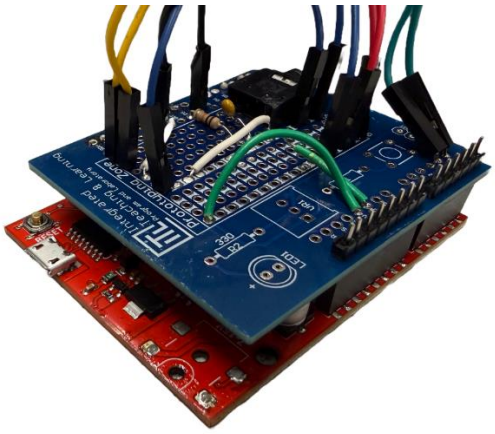
Our circuit has a user place their hands in front of two distance sensors, one vertical and one horizontal. The data is then interpreted by our microcontroller and sent out to our lcd screen and speaker.

5.2. Circuits and Components



Depending on the needs of the speaker, any attempt to reproduce our design might need an external amplifier between the 10k ohm resistor and the speaker itself. In our design, we ended up using the Fosi Audio V3 Stereo Amplifier

5.3. Protoboard Design



Our protoboard matches our electrical circuit diagram exactly, and has header pins soldered into the sides to allow connection to our Arduino uno. All wires coming out of the protoboard go directly to distance sensors, LCD, or the amplifier.

5.4. Electrical Function

In section 6, we will discuss how the microcontroller determines what signal to output to our speaker and LCD. In this section, we will focus on the electrical path our signal follows after being sent out.

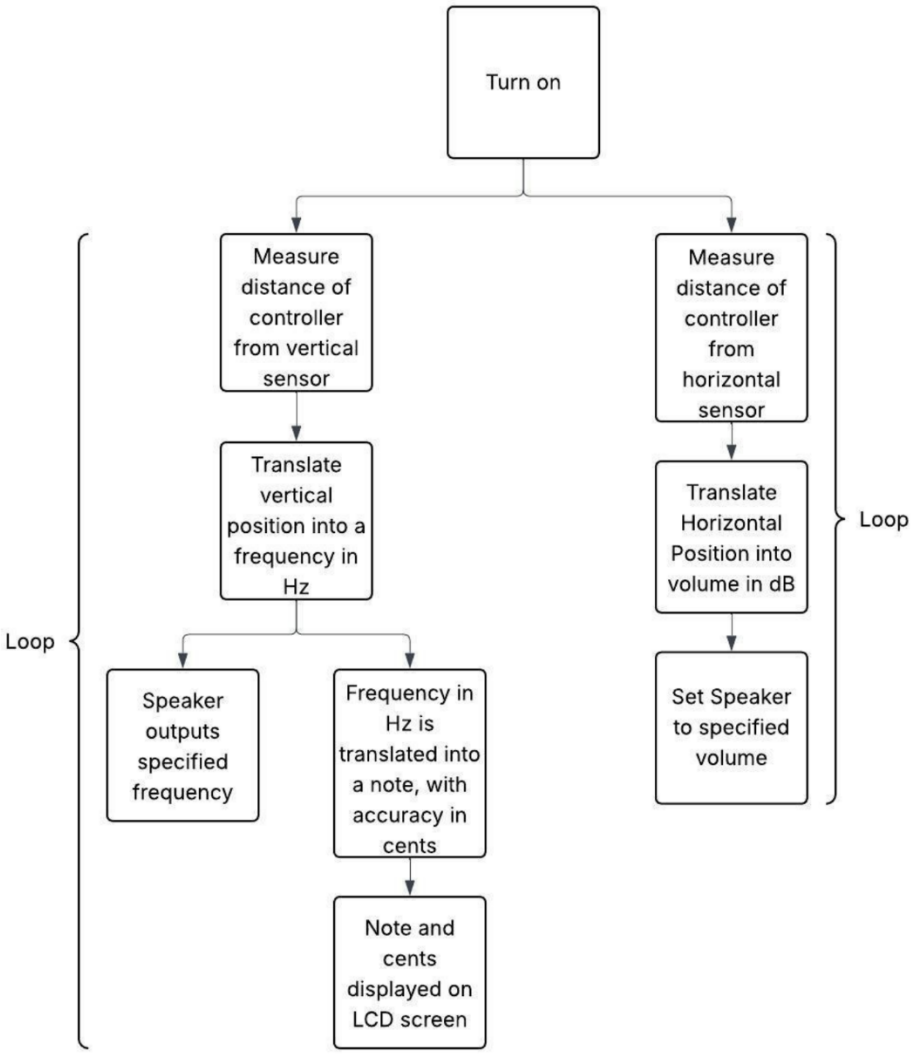
The electrical path of the LCD is very simple. For power, the LCD is connected to the 5V and Ground pins. For data, the SDA pin of the LCD is connected to A4, while the SCL pin is connected to A5.

The electrical path of the speaker is a bit more complex. After a signal is sent out by the D7 pin, it is sent through our low pass filter consisting of a 22nF capacitor in series with a 10k ohm resistor. This signal was then sent to a 3.5mm headphone socket, which allowed us to use a 3.5mm to RCA cable. This RCA cable was then plugged into our external speaker amp, and finally the amp outputs our signal into the speaker which creates the audio we hear.

6. Software Design

The main purpose of the software in our project is first to operate distance sensors and collect measurements, second to translate these measurements into frequencies and amplitudes for our sound output, and lastly to output these values to both our amplifier/speaker as well as our LCD display. Since our code runs on an Arduino Uno, we faced challenges throughout the project with getting all three parts of the code to function fast enough to prevent interference of the sound output due to a lack of processing power.

6.1. Pseudo Code; Flow Charts



This flowchart shows the basic functionality of our code. As is shown in the chart, our code is almost completely separated into two sections, one for the frequency and one for the amplitude. These two sections each operate on one distance sensor which sends our an ultrasonic pulse by turning on a shared trigger pin. The sensors then take turns receiving pulses back, since the Arduino uno doesn't allow one trigger pulse to give readings for two sensors at once. The time it takes for the pulse to return is then converted to a distance using a formula with the speed of sound.

Once the distance of the players hand from each sensor has been determined, we use a basic map function to convert distance to frequency and amplitude values.

Once we have our desired frequency and amplitude, we use functions belonging to a music library named Mozzi. Mozzi allows our code to output a smooth sine wave, which allows for a smoother sound than the square waves that digital pins would normally output. We access Mozzi with a frequency setter function and an amplitude setter function, which then change

the characteristics of the sine wave being output. We can then send this sine wave through our audio jack and then to our amplifier and speaker.

Lastly, we send our frequency value to our LCD screen through I2C output, using an I2C library named liquid crystal. We then use an LCD print function built into the library to update our LCD to display the most recent note.

Lastly, we implemented code for smoothing of audio, to prevent harsh jumps. This involves playing frequencies between the most two recent measured frequencies allowing for a smoother, more theremin like sound instead of a choppy output.

6.2. Programming issues

The main issue we faced throughout our project was maintaining a consistent sound output while performing other functions to take new readings and update the LCD in the background. This was an issue because the Arduino Uno is a single core processor, meaning that it can perform exactly one action at any time. This means that every time we wanted to perform a second action, our sound output was cut out for milliseconds. The main obstructors to our sound output were the functions pulseIn (takes readings for distance sensors) and lcd.print (updates the text on our LCD). PulseIn was an issue because the readings can only be taken as fast as sound can return to the distance sensor after sending out a pulse, which may seem fast, but if taken too often it becomes noticeable when the sound cuts off. Similarly, the LCD was an obstruction because it doesn't update as fast as would be ideal, so sending the data to print takes time every time the function loops.

This was an issue we faced throughout the semester, and in the beginning it prevented any form of smooth audio. This is what led to the implementation of the Mozzi engine to help with smoother output and updates. We found the Mozzi engine through AI, which recommended that we use the engine in place of the normal tone functions Arduino uses. This was the most significant use of AI in our project, as it recommended the Mozzi engine, and then also explained how it worked and helped us to write functions using Mozzi to update our frequency and amplitude.

This solved our issues with choppy sound output until it was time to implement the LCD screen. We struggled to find an LCD updating method that would have minimal interference with the sound. Serial monitor would slow down our code too much, so a serial connection to the LCD was not an option. We once again used AI to consult during this process, and after testing multiple types of LCDs with different connections, we settled on an I2C connection using SCL and SDA pins, combined with the liquid crystal library. This allowed us to use the lcd.print function, which we found to be the least disruptive of our sound output.

7. Mechanical Design

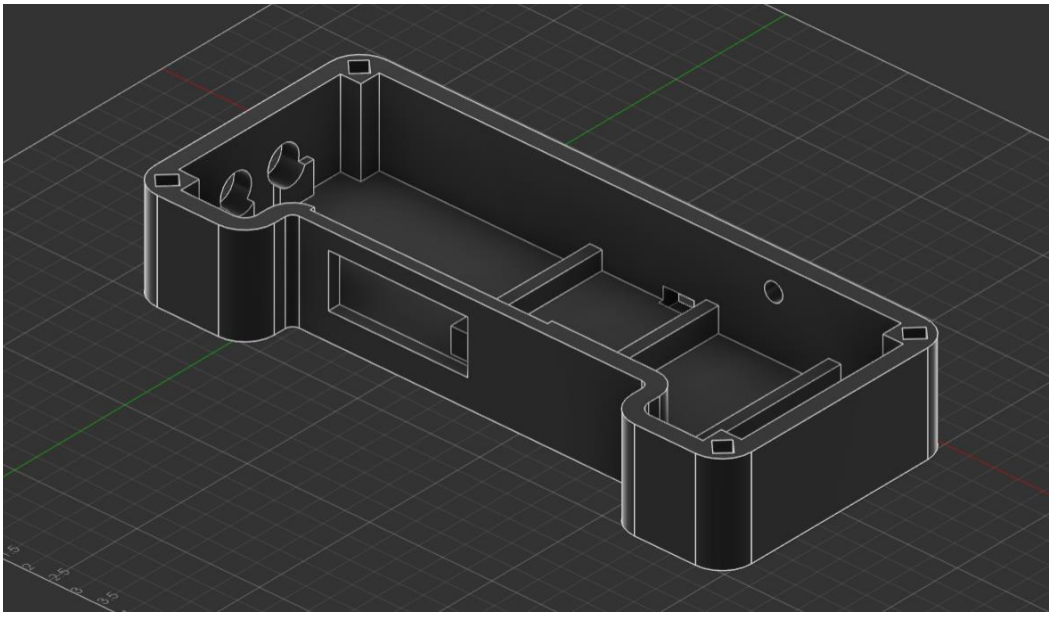
Our mechanical design consists of a plastic theremin box/electronic housing and wooden lid, connected with screws. We used AutoDesk Fusion360 to design and 3D model the theremin base, which was then 3D printed using the printers in the ITLL with basic PLA (black).

We manufactured the wooden lid in the machine shop of the ITLL, sourcing our screws and nuts here as well.

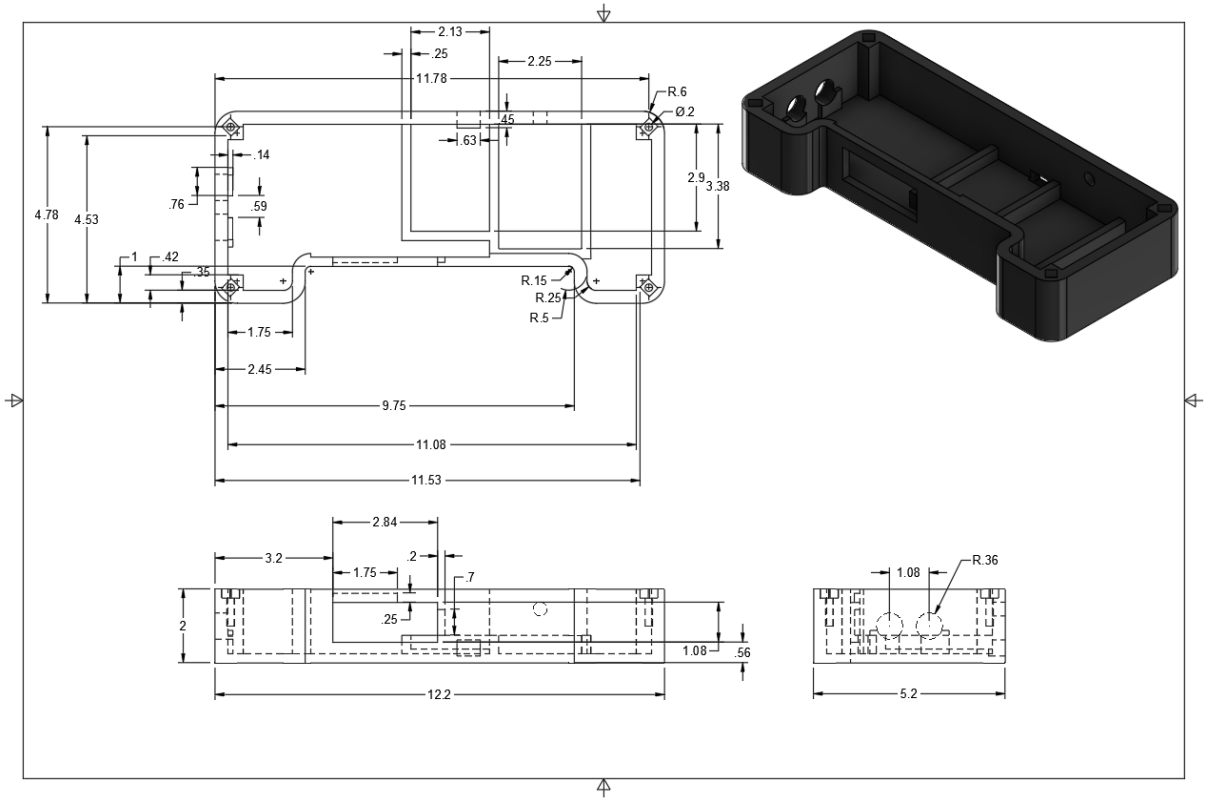
7.1. Mechanical Components

Our mechanical components include:

- The 3D printed box
    - LCD screen window
    - Distance sensor port (volume)
    - Arduino and breadboard housing
    - Nut trap and screw holes
    - Amp cable access port
    - USB power cable access port
- The wooden, stained lid
    - Distance Sensor port (pitch)
- 4 screws
- 4 nuts



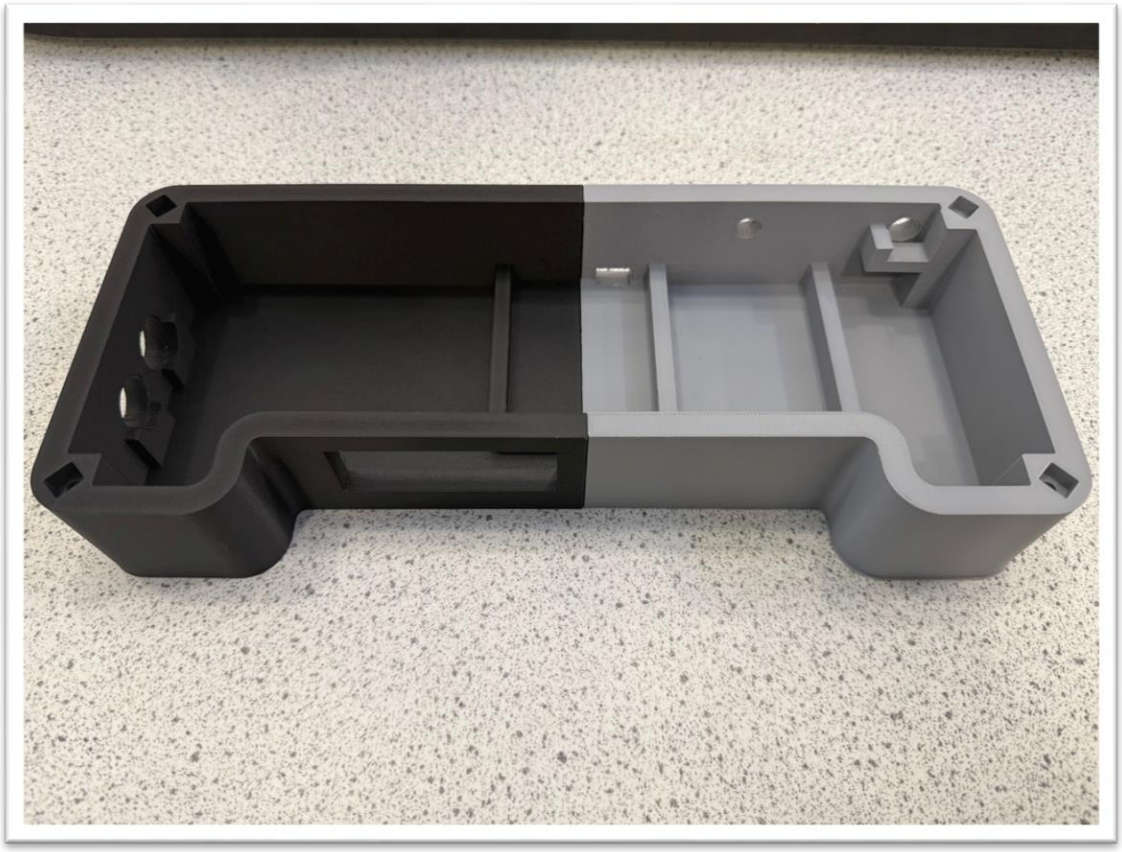
Final iteration of our 3D model of the theremin box in Fusion360



Dimensioned drawing of 3D model - all dimensions in inches, 1:2 scale

7.2. Fabrication and Assembly

The 3D printed box underwent several iterations, and we printed 3 of these iterations to test tolerances and fitting. The print bed size of the printers in the ITLL were too small for the full box design, so for preliminary iterations, we split the model in half and printed them separately. We then glued these halves together.



Third iteration of our theremin box

For our final iteration of the design, we printed the full 3D model using the large printer (Prusa XL) in the ITLL. This allowed the print to be a single piece, which looked best for our final assembly.

- Overview of wooden lid process, including wood stain

We glued the 4 nuts into the nut traps in the theremin base. This allowed us to remove/attach the wooden lid from the base easily to access electronics. The nuts allowed the screws to fit securely in the screw holes, but only 2 of our screw holes in the wooden lid lined up with those in the 3D print. So, we glued 2 screw caps in the screw holes that did not line up for consistency/aesthetics.



Final assembly of theremin

7.2.1. Tools

Software Tools:

- AutoDesk Fusion360

Manufacturing Tools:

- Prusa MK4S 3D printer
- Prusa XL 3D printer
- Wood machines (chop saw, band saw, drill press, and belt/orbital sanders)

7.3. Mechanical Functions

To operate the Accessible Theremin:

- User places right hand above the distance sensor imbedded in the wooden lid
- User places left hand to the left of the distance sensor imbedded in the 3D printed base
- Moving the right hand higher above the sensor increases pitch, lowering your hand lowers the pitch
- Moving the left hand further from the box increases volume, moving it closer decreases volume



Example photo of someone operating a theremin

To remove the wooden lid, use a screwdriver to unscrew the two full screws in the top left and bottom right corners (the other two corners are just screw caps and are not connected to the base). To attach the lid, place it on top of the base and line up the top left and bottom right screw holes in the lid with those on the base, then screw in the two screws.

8. Testing and Analysis

8.1. Electrical Testing

Objective: Verify the theremin displays the correct note on the LCD screen

Procedure: Output a constant tone from the theremin and place a phone with the Pitch Checker app nearby. Check to see if the frequency displayed on the screen matches the note read by the Pitch Checker app. Repeat this process for 10 different tones.

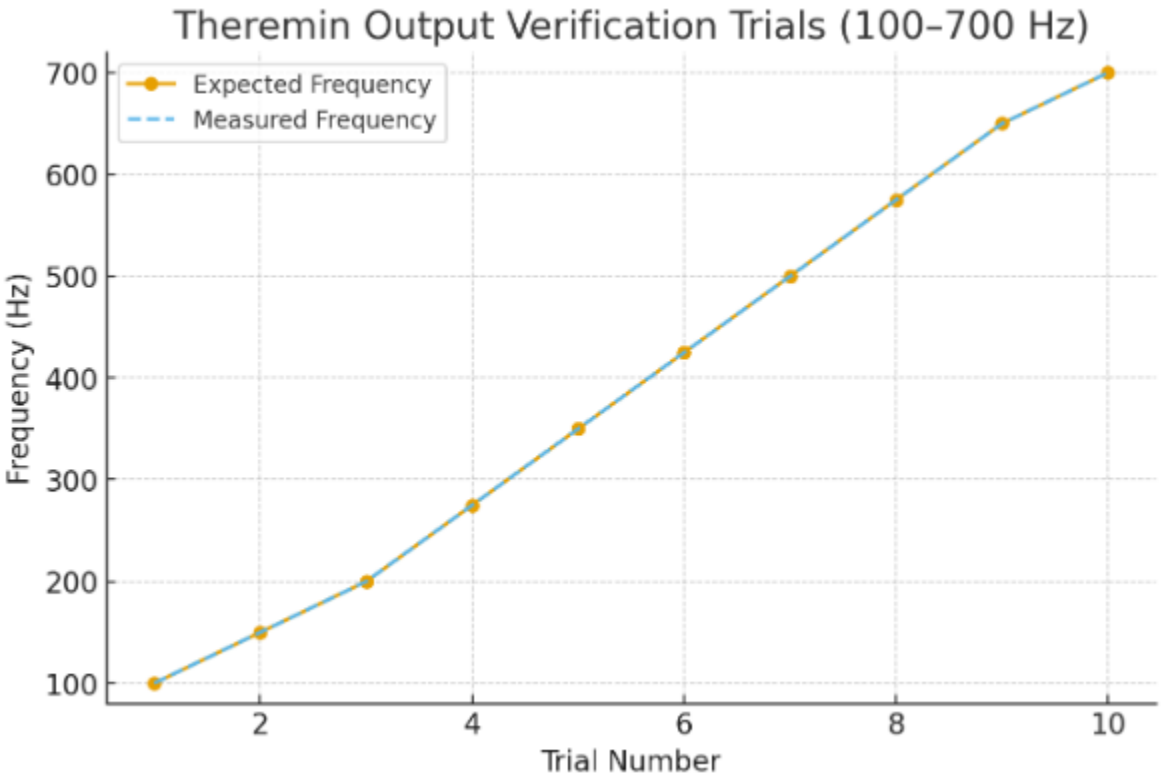
Pass: The reading on the app matches the note displayed on the screen for all trials

Fail: The screen does not display the correct note for any trial

8.1.1. Qualitative Analysis

Verified that the displayed frequency matched the reading on the Pitch Checker app for all trials, meaning the LCD screen displays the same note/frequency value that is produced by the speakers for any tone played on the theremin.

8.1.2. Quantitative Analysis



8.2. Mechanical Testing

Objective: Ensure tone output is smooth

Procedure: A phone with the Pitch Diagram app is placed near the theremin. The user moves their right hand slowly up and down one time above the distance sensor that controls pitch, keeping their left hand stationary beside the volume sensor. The user then moves their right hand quickly up and down two times, then once more slowly. The tone output is graphed over time using the Pitch Diagram app.

Pass: The pitch graph is smooth, continuous arcs for both the slow and fast hand movements

Fail: The pitch graph is choppy/not continuous with jumps, or the arcs aren't smooth and consistent

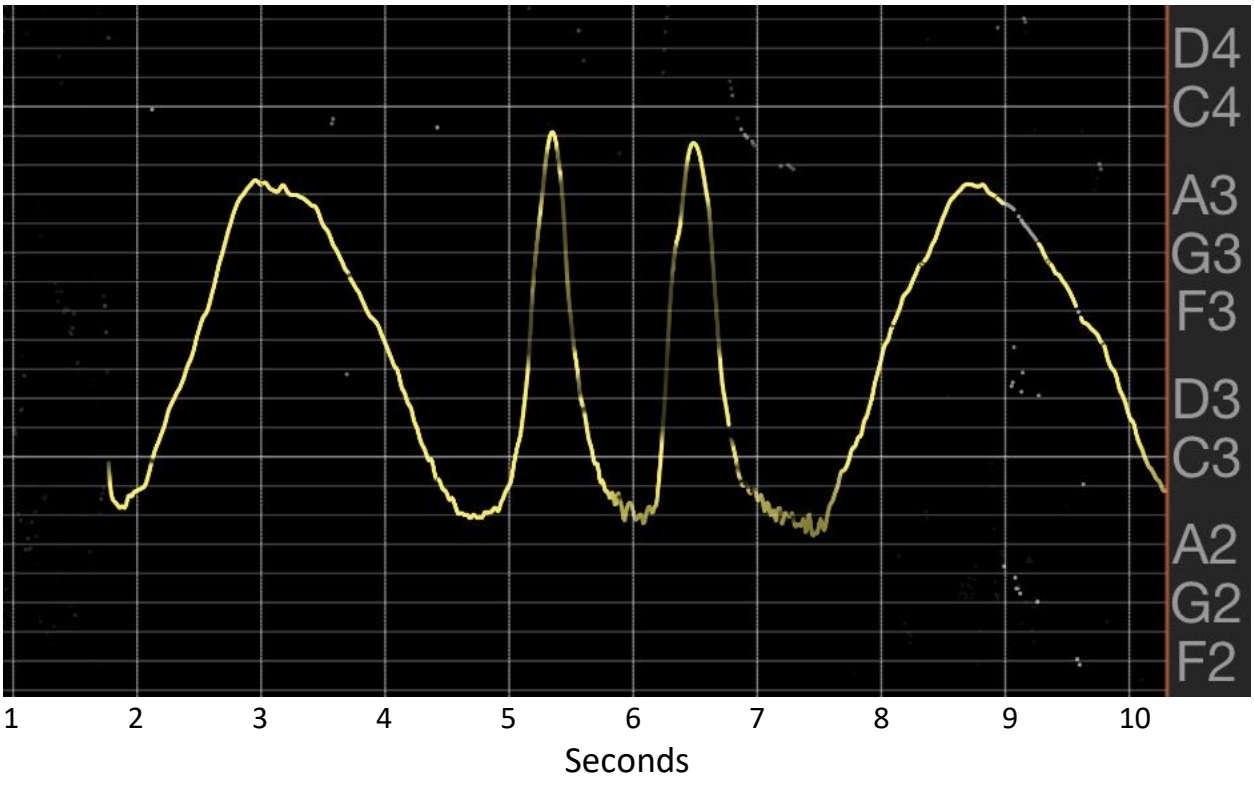
8.2.1. Qualitative Analysis

Verified that the graphed wave is smooth for both slow and quick hand movements, consistent across all trials. The graphed arcs consist mainly of a single, continuous line and do not include an excess of jagged waves.

By listening to the trials, we verified the produced sound was smooth when changing tone.

8.2.2. Quantitative Analysis

Pitch



8.3. Overall Results of Testing

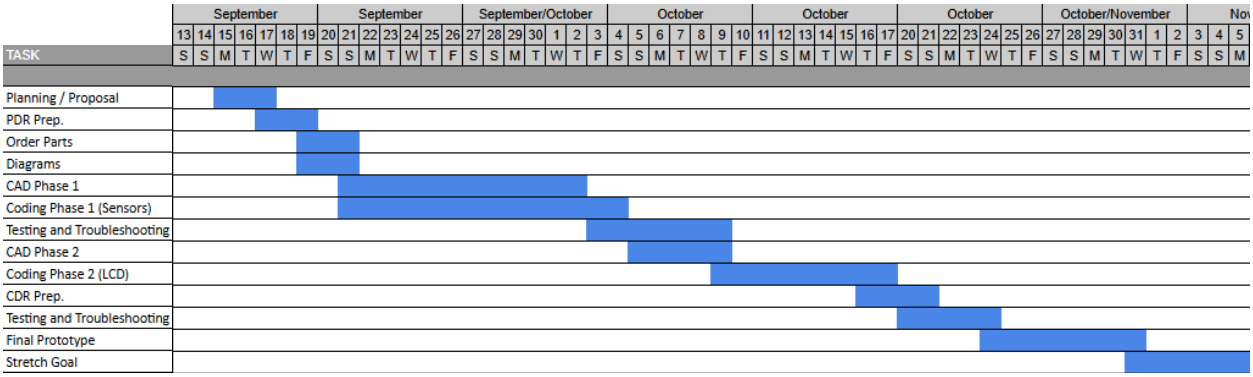
The electrical test showed the accuracy of the displayed frequency on the LCD screen, ensuring that the accessible theremin is an easier experience for the user by providing a visual of the current note that is being played, removing the ambiguity that other theremins have.

The mechanical test showed the smoothness and “gliding” sound produced when the user moves their hands and plays the instrument. This ensures the accessible theremin sounds like a traditional theremin, and that the experience/sound is the same despite the use of distance sensors.

Our testing was successful and the results of each test were able to be replicated across multiple trials.

9. Timeline and Bill of Materials.

9.1. Gantt Chart



At the beginning of our project, our group would have multiple people working together on a single task. This set us behind during the CAD Phase 1 and Coding Phase 1, and we took more time than we would have liked on these tasks. However, we learned to divide our labor better and have one person work on one task. This allowed us to work much faster and get back on target.

Additionally, we started working outside of our lab period more during CDR Prep, Testing and Troubleshooting, and Final Prototype. This ensured that we were able to stay on track and fully complete our project before Expo.

9.2. Final Bill of materials (BOM)

Material/ Component	Unit Cost	Quantity	Total Cost
PLA (in house)	$0.05/g	~0.21kg	$4 (provided)
Wood Slab	~$3.00	1	$3 (provided)
Wood Stain	$5.00	1	$5 (provided)
LCD Screen	$30.50	1	$30.50
Distance Sensors	$7.00	2	$14
RedBoard (Arduino)	$22.00	1	$22
Bread Board	$9	1	$9
Speakers	~$20.00	1	$20 (provided)
Electronics	~$3.50	1	$3.50 (provided)
Fosi Audio V3 (Amp)	$88.00	1	$88
#10-24 UNC Screws	$0.25	4	$1 (provided)
#10-24 UNC Nuts	$0.25	4	$1
		Final Cost (complete):	**$201**
		Final Cost (out of pocket):	**$164.50**

Note: “Provided” means that this material/component was either already owned by our group or supplied by the school with no additional cost.

10. Conclusion

Our theremin succeeds in being a more accessible alternative to a traditional theremin. The screen implemented into our design tells the user the exact frequency they are playing, creating a precise and unambiguous aid for the player. Additionally, using distance sensors allows our project to be much cheaper for our potential clients. Despite this change, our theremin still works and sounds like a traditional theremin. Our team was able to create a finished product that not only functions as intended, but one that looks and sounds beautiful and market ready.

This project helped all of us learn new engineering skills, increase our expertise in many areas, and improve as team members. There is also still room for our project to grow and improve.

10.1. Lessons Learned

Software Challenges – One of our greatest coding challenges was the problem of producing a smooth, clean tone from the distance to frequency values. Another challenge we ran into was being able to display the note name on the LCD screen without causing disturbance in the sound output. These software hurdles allowed our team to improve our coding skills, which was

one of our weakest subject areas going into the project. We all learned more about the limits of an Arduino's single core processor, as well as the usefulness of coding libraries.

Exterior Design Challenges – The process of finding precise yet usable tolerances was a challenge for our team, but it helped us learn what tolerance sizes were most effective, as well as the most successful ways to 3D model designs with many different types of fitting, ports, and housing.

We also learned more about the process of manufacturing/fabricating wood pieces. We gained lots of experience with the machinery and tools in the wood shop. We also learned cutting, sawing, drilling, sanding, and staining techniques.

Team Growth – The biggest lesson we will take away from this project is learning to communicate and understand other group members. Our group ran into problems with communication, and we often failed to reach agreement or compromise. We did not always operate as a team, but learning to properly communicate with each other allowed us to collaborate successfully and produce an impressive final product.

We have each gained crucial experience in working with others as a team, and we will be able to better avoid conflicts and diffuse negative situations in future teams throughout our engineering careers.

10.2. Next Steps or Future Work

Despite all our successes, there are still goals we wish to accomplish with our project. First, we would like to add a second Arduino to our theremin to increase processing power. This will enable us to have our LCD screen display the actual note, rather than the frequency, without disturbance to our produced sound.

The additional Arduino will also enable us to implement the "cents" measure on the screen for a more accurate pitch reading. "Cents" is a unit that measures how out of tune a pitch is relative to certain note (i.e. how "sharp" or "flat" a note is).

Finally, we plan to include the speaker and amplifier system fully inside the electronic housing. This means finding a smaller amp and speaker or increasing the size of the printed box. We will also include the power input in the electronic housing as well by using batteries. This will make the theremin wire-free and easily portable. Our goal is to make our theremin completely self-contained with no external components.